Btc4Stable Small V2

Security Review

Scope: `saabfi-btc4stable-small-v2/contracts`

Review Commit: `fd1c527`

31.08.2026 - 02.09.2026

Conducted by :

Synthrax, Security Researcher

Contents

1 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where I try to find as many vulnerabilities as possible. I can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

2 Protocol Overview

BTC4Stable is a cbBTC-collateralized USDC microloan protocol deployed on Base (L2). cbBTC holders post Bitcoin collateral to take USDC microloans ranging from $1 to $1,000, paying a fixed upfront commitment fee. The protocol is non-liquidating - borrowers either repay to reclaim their cbBTC, or forfeit it to the Treasury on default.

V2 replaces the V1 30-day bullet loan with a 60-day term split into two installments: a billing checkpoint at day 30 requiring half of the outstanding principal plus all accrued interest, and a final payment at maturity, each followed by a 7-day grace period. The one-time dynamic commitment fee is replaced by a fixed origination fee routed directly to the company Safe, plus continuous per-term interest priced by a utilization-driven model in `LendingPool`. Repayment interest is routed to a `TokenJar` and drained by a `Releaser`, which splits it between the Safe and a SAABFI buyback-and-burn executed on Uniswap v4.

3 Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - involves a small loss of funds or affects a core functionality of the protocol.
- **Low** - encompasses any unexpected behavior that is non-critical.

3.2 Likelihood

- **High** - a direct attack vector; the cost is relatively low compared to the potential loss of funds.
- **Medium** - only a conditionally incentivized attack vector, with a moderate likelihood.
- **Low** - involves too many or unlikely assumptions; offers little to no incentive.

3.3 Actions required by severity level

- **High** - client **must** fix the issue.
- **Medium** - client **should** fix the issue.
- **Low** - client **could** fix the issue.

4 Findings Summary

ID	Title	Severity
[L-01]	Collateral ratio and maximum interest are never validated against each other	Low
[I-01]	Interest accrues indefinitely past maturity	Informational
[I-02]	`MISSED_FINAL_PAYMENT` is unreachable once the first installment is missed	Informational

5 Detailed Findings

5.1 Low Findings

5.1.1 [L-01] Collateral ratio and maximum interest are never validated against each other

Severity: Low **Status:** Acknowledged, V2 will remain at 80% LTV with a 16% maximum rate, where day-37 debt remains below collateral. Higher-LTV, rate-aware design is deferred to V3.

File(s): `OriginationPolicy.sol`, `LendingPool.sol`

Description: `setCollateralRatioBps()` validates `newBps` only against `MIN_COLLATERAL_RATIO_BPS` / `MAX_COLLATERAL_RATIO_BPS`, and `setDynamicInterestConfig()` validates `maxInterestBps` only against `MAX_BPS`. No code path checks that the posted collateral covers principal plus maximum accruable interest. The point at which a loan goes underwater is `60 * (collateralRatioBps - 10000) / maxInterestBps` days.

Impact: The protocol is non-liquidating, so an underwater position is never corrected. A loan originated at `MIN_COLLATERAL_RATIO_BPS` (10526) with `MAX_INTEREST_BPS` (1600) quoted becomes cheaper to default on than to repay after 19.7 days, before the first installment is due.

Recommendation(s): Add checks in both setters to verify that the loan won't go underwater before the default.

5.2 Informational Findings

5.2.1 [I-01] Interest accrues indefinitely past maturity

Severity: Informational **Status:** Acknowledged, works as intended.

File(s): `Microloans.sol`

Description: `_settleAccrual()` accrues to `settlementTimestamp` with no clamp at `startTimestamp + MICRO_LOAN_MATURITY`, and `_settleAccrualMemory()` does the same. The rate is denominated per 60 days and `INTEREST_ACCRUAL_DENOMINATOR` is derived from `MICRO_LOAN_MATURITY`, but nothing bounds the elapsed term. Default is not automatic: it requires an external call to the `restricted` `LoanManager.processDefault()`, so a loan remains `ACTIVE` and accruing until a keeper acts.

Impact: A borrower repaying at day 67, the earliest point the loan is defaultable, owes 112% of the quoted 60-day rate, with no ceiling beyond that. Total cost becomes a function of off-chain keeper latency rather than the loan terms. This is also the mechanism behind the collateral-ratio finding above: unbounded accrual is what allows debt to exceed collateral at ratios that look adequate at maturity.

Recommendation(s): Clamp `settlementTimestamp` to `startTimestamp + MICRO_LOAN_MATURITY` inside `_settleAccrual()` and `_settleAccrualMemory()`, or introduce an explicit, bounded late-payment charge so post-maturity cost is a stated term rather than an emergent one.

5.2.2 [I-02] `MISSED_FINAL_PAYMENT` is unreachable once the first installment is missed

Severity: Informational **Status:** Acknowledged, works as intended.

File(s): `Microloans.sol`

Description: `_defaultReasonMemory()` evaluates the first-bill branch before the maturity branch and returns from inside it, so the maturity branch is never reached for a loan whose first bill is unsatisfied. A borrower who missed the day-30 installment and then also passed `finalDefaultTimestamp` is always reported as `MISSED_FIRST_INSTALLMENT`; `MISSED_FINAL_PAYMENT` can only ever be returned for a loan that satisfied the first bill.

Impact: Cosmetic. `unpaidPrincipal` is derived from storage independently of the reason code, so collateral seizure and the pool write-off are unaffected. Off-chain consumers of `LoanDefaulted` and `checkDefaultConditions()` cannot distinguish a borrower who missed one deadline from one who missed both, which understates delinquency in reporting.

Recommendation(s): Evaluate the maturity branch first, or add a distinct reason for the both-missed case.